Monoids: an introduction to lawful structures

George Wilson

george@wils.online

13th October 2026

Outline

Let's look at:

- Basic Haskell syntax
- A simple example type class
- The Monoid type class (including laws)
- Examples of monoid instances
- Examples of functions defined for all monoids
- Why laws are helpful

Values and types

```
x :: Integer
x = 5 + 10
```

Strings and concatenation

```
bfpg :: String
bfpg = "BFPG"

greeting :: String
greeting = "Hello, " ++ bfpg ++ "!"
```

Strings and concatenation

```haskell
bfpg :: String
bfpg = "BFPG"

greeting :: String
greeting = "Hello, " ++ bfpg ++ "!"
```

```
> greeting
-- "Hello, BFPG!"
```

Functions on strings

```
greet :: String -> String
greet name = "Hello, " ++ name ++ "!"
```

Functions on strings

```haskell
greet :: String -> String
greet name = "Hello, " ++ name ++ "!"
```

```haskell
> greet bfpg
-- "Hello, BFPG!"
```

Custom datatypes

```
data MeetupGroup =
  MeetupGroup {
    name :: String,
    url :: String,
    description :: String
  }
```

Custom datatypes

```haskell
data MeetupGroup =
  MeetupGroup {
    name :: String,
    url :: String,
    description :: String
  }
bfpgMeetup :: MeetupGroup
bfpgMeetup =
  MeetupGroup {
    name = "Brisbane Functional Programming Group",
    url = "https://luma.com/bfpg",
    description = "a bunch of lambda heads"
  }
```

Custom data types

```
data TrafficLight
  = Red
  | Yellow
  | Green
```

Type classes

```
class Show a where
  show :: a -> String
```

Type classes

```
class Show a where
  show :: a -> String

instance Show TrafficLight where
  show tl =
    case tl of
      Red -> "Red"
      Yellow -> "Yellow"
      Green -> "Green"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"
```

Functions with type class constraints

```
greet :: Show a => a -> String
greet s = "Hello, " ++ show s ++ "!"
```

```
> greet Yellow
-- "Hello, Yellow!"

> greet 5
-- "Hello, 5!"

> greet (++)
-- Error: No Show instance for
--          (String -> String -> String)
```

We've seen Haskell expressions, data types, functions, and type classes.

Any questions at this stage?

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
instance Monoid String where
  mappend s t = s ++ t
  mempty = ""
```

Monoid

```
class Monoid m where
  mappend :: m -> m -> m
  mempty  :: m
instance Monoid String where
  mappend s t = s ++ t
  mempty = ""
-- mappend as an infix operator, for convenience
x <> y = mappend x y
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Right identity:

```
-- for all x.
x <> mempty == x
```

Monoid laws

Left identity:

```
-- for all x.
mempty <> x == x
```

Right identity:

```
-- for all x.
x <> mempty == x
```

Associativity:

```
-- for all x y z.
x <> (y <> z) == (x <> y) <> z
```

Monoid laws specialised to strings

Left identity:

```
-- for all x.
"" ++ x == x
```

Right identity:

```
-- for all x.
x ++ "" == x
```

Associativity:

```
-- for all x y z.
x ++ (y ++ z) == (x ++ y) ++ z
```

Pretty printing

Imagine we are trying to pretty print LISP s-expressions

```
expr =
  openParen <> "+" <> space <> "3"
    <> space <> "4" <> closeParen

-- (+ 3 4)
```

Pretty printing

1: Factor out the parens to a helper function

```
parens x = openParen <> x <> closeParen



expr =
  parens ("+" <> space <> "3" <> space <> "4")


-- (+ 3 4)
```

Pretty printing

2: Factor out the space separation to a helper function

```
parens x = openParen <> x <> closeParen

spaceSep list = ... -- omitted for brevity

expr =
  parens (spaceSep ["+", "3", "4"])


-- (+ 3 4)
```

Extracting a helper function usually depends on associativity

Extracting helper functions is awesome

Therefore associativity is awesome!

What are some other types that fit monoid?

Monoid laws specialised to addition

Left identity:

```
-- for all x.
0 + x == x
```

Right identity:

```
-- for all x.
x + 0 == x
```

Associativity:

```
-- for all x y z.
x + (y + z) == (x + y) + z
```

Monoid laws specialised to multiplication

Left identity:

```
-- for all x.
1 * x == x
```

Right identity:

```
-- for all x.
x * 1 == x
```

Associativity:

```
-- for all x y z.
x * (y * z) == (x * y) * z
```

Integer instances?

We can't have multiple instances like this in Haskell

```
instance Monoid Integer where
  mappend x y = x + y
  mempty = 0

instance Monoid Integer where
  mappend x y = x * y
  mempty = 1
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }

instance Monoid Sum where
  mappend (Sum x) (Sum y) = Sum (x + y)
  mempty = Sum 0
```

Integer instances with newtype wrappers

```haskell
newtype Sum = Sum { unwrapSum :: Integer }
instance Monoid Sum where
  mappend (Sum x) (Sum y) = Sum (x + y)
  mempty = Sum 0

newtype Product = Product { unwrapProduct :: Integer }
instance Monoid Product where
  mappend (Product x) (Product y) = Product (x * y)
  mempty = Product 1
```

Now we have a bunch of monoids.
What can we do with them?

Derived functions

```
mconcat :: Monoid m => [m] -> m
```

Derived functions

```
mconcat :: Monoid m => [m] -> m
```

```
> mconcat [Sum 1, Sum 2, Sum 3, Sum 4]
-- Sum 10
```

Derived functions

```
mconcat :: Monoid m => [m] -> m
```

```
> mconcat [Sum 1, Sum 2, Sum 3, Sum 4]
-- Sum 10
> mconcat (map Sum [1, 2, 3, 4])
-- Sum 10
```

Derived functions

```
mconcat :: Monoid m => [m] -> m
```

```
> mconcat [Sum 1, Sum 2, Sum 3, Sum 4]
-- Sum 10
> mconcat (map Sum [1, 2, 3, 4])
-- Sum 10
> mconcat (map Sum [])
-- Sum 0
```

that's mempty

Derived functions

Is mconcat implemented using a left fold or a right fold?

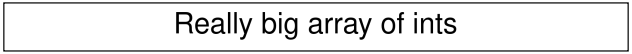
```
Sum 1 <> (Sum 2 <> (Sum 3 <> (Sum 4 <> mempty)))
```

or

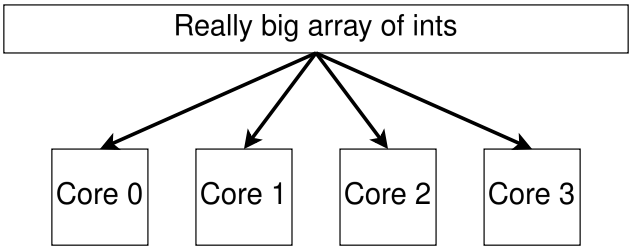
```
(((mempty <> Sum 1) <> Sum 2) <> Sum 3) <> Sum 4
```

Answer: We don't have to care, because of associativity

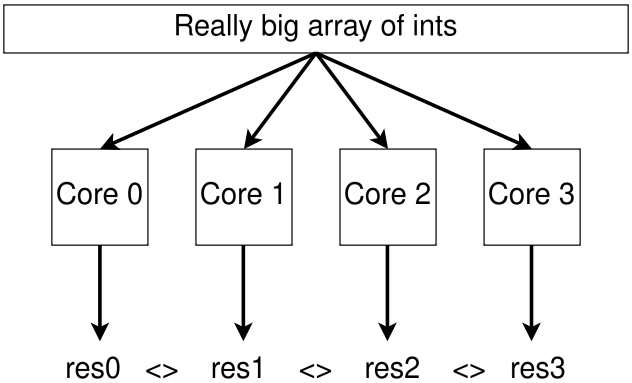
Parallelism



Parallelism



Parallelism



Putting Monoids together

```
data Mean =
  Mean {
    total :: Integer,
    count :: Integer
  }
```

Putting Monoids together

```haskell
data Mean =
  Mean {
    total :: Integer,
    count :: Integer
  }
instance Monoid Mean where
  mappend (Mean t1 c1) (Mean t2 c2) =
    Mean (t1 + t2) (c1 + c2)
  mempty =
    Mean 0 0
```

Putting Monoids together

```haskell
data Mean =
  Mean {
    total :: Integer,
    count :: Integer
  }
instance Monoid Mean where
  mappend (Mean t1 c1) (Mean t2 c2) =
    Mean (t1 + t2) (c1 + c2)
  mempty =
    Mean 0 0
makeMean :: Integer -> Mean
makeMean num = Mean { total = num, count = 1 }

getMean :: Mean -> Double
getMean (Mean total count) =
  fromIntegral total / fromIntegral count
```

Means

```
> mconcat (map makeMean [1,2,3,4,5])
-- Mean { total = 15, count = 5 }
```

Means

```
> mconcat (map makeMean [1,2,3,4,5])
-- Mean { total = 15, count = 5 }
> getMean (mconcat (map makeMean [1,2,3,4,5]))
-- 3.0
```

Generalised mconcat

```
class Foldable f where
  foldMap :: Monoid m => (a -> m) -> f a -> m
```

Generalised mconcat

```
class Foldable f where
  foldMap :: Monoid m => (a -> m) -> f a -> m
instance Foldable [] where
  foldMap a2m list = mconcat (map a2m list)
```

Generalised mean

```
mean :: Foldable f => f Integer -> Double
mean ints = getMean (foldMap makeMean ints)
```

This function takes the mean of a structure, without caring about the "shape" of the structure

Trees

```haskell
data Tree a = Leaf a | Branch (Tree a) (Tree a)
```

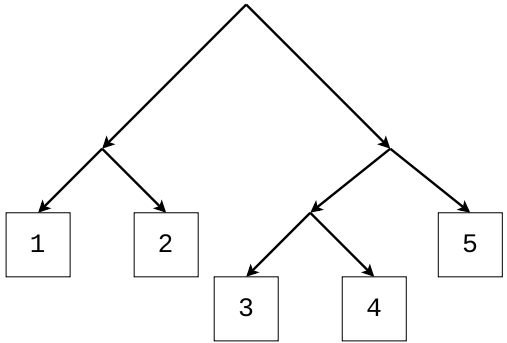
Trees

```haskell
data Tree a = Leaf a | Branch (Tree a) (Tree a)

instance Foldable Tree where
  foldMap a2m tree =
    case tree of
      Leaf a -> a2m a
      Branch t u ->
        foldMap a2m t <> foldMap a2m u
```

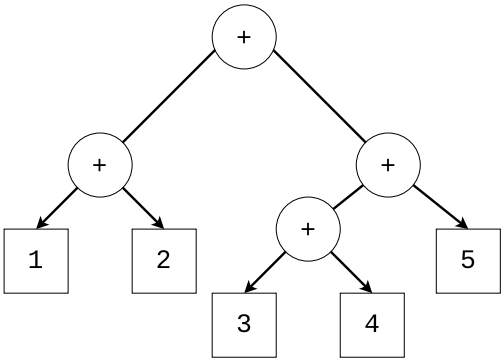
Trees

```
myTree =
  Branch
    (Branch (Leaf 1) (Leaf 2))
    (Branch
      (Branch (Leaf 3) (Leaf 4))
      (Leaf 5)
    )
```



Trees

```
foldMap Sum myTree
```



There are many more foldable structures

Outline

We've seen:

- Monoid typeclass and its laws
- multiple instances of monoid (String, Sum, Product)
- Laws allowed us to extract helper functions
- combining instances to make new instances (Mean)
- Functions that work on all monoids (mconcat, foldMap)
- Laws allowed us to concatenate structures regardless of their shapes

There is much more to monoids.

There are many other sturctures with laws.

Thanks for listening!

spaceSep

```
spaceSep [] = mempty
spaceSep (x:[]) = x
spaceSep (x:y:rest) = x <> space <> spaceSep (y:rest)
```